

Universidad de Guadalajara
Centro Universitario de Ciencias Exactas e Ingenierías

Laboratorio Abierto: Construcción

Manual de trabajo y gestión del proyecto

Profesor responsable

Dr. Jairo Cain Sánchez Estrada

Ciclo escolar 2026

Índice

1. Propósito del laboratorio	3
2. Forma general de trabajo	3
3. Organización del proyecto	3
3.1. Roles	4
3.2. Selección del Project Leader	4
4. Herramientas obligatorias	4
4.1. Creación de la cuenta de GitHub	5
4.2. Visibilidad del repositorio	5
4.3. Repositorio del proyecto	5
4.4. El README principal	6
5. Design Baseline	6
5.1. Contenido	6
5.2. README de la Design Baseline	7
6. Primera versión congelada del proyecto	7
7. Flujo de trabajo obligatorio en GitHub	7
7.1. La rama main	7
7.2. GitHub Issues	8
7.3. Branches	8
7.3.1. Ejemplos	8
7.4. Commits	9
7.5. Pull Requests	9
8. GitHub Project	9
9. SPRINT I: Planificación de la construcción	10
9.1. Documentos obligatorios del Sprint I	10
9.2. Documento 1: Project Charter	10
9.3. Documento 2: Work Breakdown Structure (WBS)	11
9.4. Documento 3: RACI Matrix	11
9.5. Documento 4: Risk Register	11
9.6. Documento 5: Project Schedule	11
10.Actividades del Sprint I en GitHub	11
10.1. Relación entre WBS, RACI y GitHub	12
11.Entrega del Sprint I	12
12.SPRINT II: Seguimiento, evaluación y adaptación	13
12.1. Seguimiento continuo	13
12.2. Reuniones del equipo	13
12.3. Revisión con el mentor	13
12.4. Project Review	14
12.5. Cambios durante la construcción	14
12.6. Documentos vivos durante el Sprint II	14
13.Entrega del Sprint II	15

14.SPRINT III: Consolidación y transición hacia Pruebas	15
14.1. Consolidación	15
14.2. Handover	16
14.3. Construction Handover Package	16
15.Versión final del Sprint III	17
16.Las tres versiones principales del proyecto	17
16.1. Resumen operativo de los tres Sprints	17
16.2. Mapa general del proceso	18
17.Reglas generales del repositorio	19
17.1. Convención recomendada de nombres	19
17.2. Qué significa “terminado”	19
17.3. Uso de evidencia	19
17.4. Bitácora de decisiones	20
18.Actores y Roles	20
18.1. Qué se espera del Project Leader	20
18.2. Qué se espera del Project Team	20
18.3. Qué se espera del mentor	21
19.Qué se espera durante una revisión	21
19.1. Trazabilidad individual	21
20.Listados de verificación	21
20.1. Sprint I	21
20.2. Sprint II	22
20.3. Sprint III	22
21.Principios del laboratorio	23
22.Mensaje final al equipo	23

1. Propósito del laboratorio

El **Laboratorio Abierto: Construcción** tiene como propósito llevar una solución previamente diseñada hacia su materialización mediante una forma de trabajo semejante, dentro de las posibilidades de un curso universitario, a la utilizada en proyectos reales de ingeniería.

La materia no consiste únicamente en "construir algo". El equipo deberá ser capaz de:

- Recibir formalmente un diseño previamente desarrollado
- Transformar dicho diseño en un plan de construcción
- Dividir el trabajo en actividades e hitos
- Asignar responsabilidades
- Establecer tiempos y recursos
- Identificar riesgos
- Ejecutar y documentar el trabajo
- Controlar modificaciones
- Utilizar control de versiones
- Demostrar quién realizó cada contribución
- Comparar lo planeado con lo realmente ejecutado
- Consolidar el estado final de la construcción
- Transferir el resultado al Laboratorio Abierto: Pruebas

Objetivo

Al finalizar la materia, el equipo no solo deberá mostrar un prototipo, software, sistema, dispositivo, proceso o producto construido. También deberá poder reconstruir documentalmente **cómo evolucionó el proyecto, quién realizó el trabajo, qué decisiones se tomaron, qué cambió y cuál es el estado real de la solución.**

2. Forma general de trabajo

La materia se organizará mediante tres etapas principales, denominadas **Sprints**:

Sprint	Propósito principal	Resultado esperado
Sprint I	Planificación de la construcción	Establecer la línea base inicial del proyecto de construcción.
Sprint II	Seguimiento, evaluación intermedia y adaptación	Contrastar lo planeado con lo ejecutado y actualizar el proyecto.
Sprint III	Consolidación y transición hacia Pruebas	Determinar el estado real de la solución y preparar su transferencia.

Los tres Sprints forman un solo proyecto continuo:

Diseño → **Planificación** → **Construcción** → **Seguimiento** → **Consolidación** → Pruebas

No se trata de tres proyectos independientes.

3. Organización del proyecto

Cada proyecto deberá funcionar como una pequeña organización de trabajo.

3.1. Roles

Participante	Rol equivalente	Responsabilidad principal
Profesor responsable	Course Supervisor / Key Stakeholder	Establece las reglas generales del laboratorio, revisa los Sprints, solicita evidencia, evalúa resultados y supervisa el cumplimiento del proceso.
Dr. investigador o mentor	Project Sponsor / Technical Mentor	Orienta técnicamente el proyecto, participa en decisiones relevantes, ayuda a resolver bloqueos y valida aspectos importantes de la construcción.
Alumno seleccionado por el equipo	Project Leader	Coordina el proyecto, organiza el trabajo, mantiene actualizada la planeación, supervisa actividades y representa al equipo en las revisiones.
Demás integrantes	Project Team	Realizan las actividades técnicas y administrativas que les hayan sido asignadas y documentan sus aportaciones.

Importante

El Project Leader **no es el alumno que debe hacer todo**. Su función principal es coordinar el proyecto y procurar que el trabajo pueda ser planeado, ejecutado, documentado y revisado.

3.2. Selección del Project Leader

Cada equipo deberá seleccionar a uno de sus integrantes como **Project Leader**.

La decisión deberá quedar documentada desde el Sprint I.

La selección puede considerar:

- Capacidad de organización
- Capacidad para dar seguimiento al trabajo
- Conocimiento general del proyecto
- Capacidad para coordinar al equipo
- Comunicación
- Disponibilidad

El rol de Project Leader no implica una calificación superior ni una autoridad absoluta sobre los demás integrantes.

4. Herramientas obligatorias

Durante el curso será obligatorio utilizar una plataforma real de control de versiones y gestión colaborativa. La herramienta principal será:

GitHub

GitHub se utilizará para:

- Alojar el proyecto
- Trabajar mediante ramas
- Mantener control de versiones
- Revisar contribuciones
- Documentar cambios
- Integrar cambios mediante Pull Requests
- Asignar actividades
- Mantener evidencia de la evolución del proyecto
- Registrar responsables
- Identificar contribuciones individuales
- Conservar versiones correspondientes a los diferentes Sprints

Advertencia

GitHub **no se utilizará solamente como almacenamiento de archivos**. Subir todos los archivos al final de un Sprint no constituye un uso adecuado del repositorio.

4.1. Creación de la cuenta de GitHub

Cada integrante deberá contar con una **cuenta individual de GitHub**.

No se permitirán cuentas compartidas.

Cada estudiante deberá:

1. crear o utilizar su propia cuenta de GitHub;
2. configurar correctamente su nombre;
3. utilizar un correo electrónico asociado a su cuenta;
4. informar al Project Leader cuál es su usuario de GitHub;
5. aceptar la invitación al repositorio del proyecto;
6. verificar que sus contribuciones aparecen asociadas a su propia cuenta.

Regla de trabajo

Cada integrante deberá realizar sus propias contribuciones utilizando su propia cuenta. Un integrante no deberá realizar commits, cargas de archivos o Pull Requests en nombre de otro integrante.

4.2. Visibilidad del repositorio

El repositorio podrá ser: **público** o **privado**.

La decisión se tomará considerando la naturaleza del proyecto.

Si existen:

- Resultados de investigación no publicados
- Información confidencial
- Propiedad intelectual
- Desarrollos susceptibles de protección
- Restricciones definidas por el mentor

el repositorio deberá mantenerse privado.

Importante

La visibilidad del repositorio será determinada por el mentor y el supervisor del laboratorio. Un repositorio privado conserva igualmente el historial de contribuciones y la trazabilidad del trabajo.

4.3. Repositorio del proyecto

Cada equipo tendrá **un único repositorio principal**.

Todos los integrantes trabajarán sobre dicho repositorio mediante sus propias cuentas.

Una estructura recomendada es:

```
README.md

/design-baseline

/project-management
  /charter
  /wbs
  /raci
  /risks
  /schedule

/docs
  /technical

/src
```

```

/hardware

/software

/cad

/bom

/tests

/evidence

```

No todos los proyectos requieren exactamente las mismas carpetas.

Por ejemplo:

- un proyecto electrónico puede utilizar `/hardware`;
- un proyecto computacional puede utilizar principalmente `/software` o `/src`;
- un proyecto mecánico puede utilizar `/cad`;
- un proyecto interdisciplinario puede utilizar varias de ellas.

La estructura deberá representar adecuadamente la naturaleza del proyecto.

4.4. El README principal

Todo repositorio deberá contener un archivo:

```
README.md
```

El README deberá permitir comprender rápidamente el proyecto.

Como mínimo deberá indicar:

- | | | |
|-----------------------|----------------------------------|--|
| ▪ Nombre del proyecto | ▪ Project Leader | torio |
| ▪ Breve descripción | ▪ Integrantes del equipo | ▪ Versión o Sprint actual |
| ▪ Objetivo general | ▪ Estado actual | ▪ Instrucciones necesarias para consultar o reproducir el proyecto cuando sean pertinentes |
| ▪ Mentor | ▪ Estructura general del reposi- | |

5. Design Baseline

Antes de iniciar formalmente la construcción deberá establecerse una **Design Baseline**.

La Design Baseline representa el estado del diseño que el equipo recibe del **Laboratorio Abierto: Diseño**.

Conceptualmente:

Design Baseline = punto técnico de partida de Construcción

5.1. Contenido

Deberá crearse la carpeta:

```
/design-baseline
```

y almacenar en ella los elementos técnicos recibidos del **laboratorio de diseño** anterior.

Dependiendo del proyecto, pueden incluirse:

- memoria o reporte de diseño
- requisitos
- diagramas
- esquemáticos
- planos
- modelos CAD
- simulaciones
- algoritmos
- código
- lista preliminar de materiales
- modelos matemáticos
- especificaciones
- documentación adicional

5.2. README de la Design Baseline

Dentro de dicha carpeta deberá existir:

```
/design-baseline/README.md
```

que incluya, al menos:

Campo	Información
Proyecto	Nombre oficial del proyecto.
Procedencia	Laboratorio Abierto: Diseño.
Fecha de recepción	Fecha en la que Construcción recibe el diseño.
Versión	Design Baseline v1.0.
Mentor	Nombre del mentor.
Project Leader	Nombre del Project Leader.
Estado del diseño	Completo, parcial, aprobado, con pendientes, etc.
Documentos recibidos	Relación de documentos y artefactos.
Pendientes conocidos	Aspectos todavía no resueltos.
Restricciones conocidas	Recursos, tecnología, materiales, tiempo, etc.

6. Primera versión congelada del proyecto

Una vez cargada y verificada la Design Baseline, deberá crearse una versión identificable mediante una etiqueta:

```
design-baseline-v1.0
```

Esta versión representa:

“Éste es el diseño recibido por el equipo antes de iniciar formalmente la construcción.”

Importante

La Design Baseline no significa que el diseño sea perfecto ni que no pueda modificarse. Su propósito es permitir identificar posteriormente qué cambios fueron necesarios durante la construcción.

7. Flujo de trabajo obligatorio en GitHub

Después de establecer la Design Baseline, el trabajo deberá seguir el flujo:

```
Issue → Branch → Commits → Pull Request → Review → Merge
```

7.1. La rama main

La rama:

```
main
```


representará el estado integrado y estable del proyecto.

Después de establecer la Design Baseline, los integrantes **no deberán desarrollar directamente sobre main**.

El trabajo deberá realizarse mediante ramas.

7.2. GitHub Issues

Una actividad relevante del proyecto deberá registrarse mediante un **GitHub Issue**.

Ejemplo:

```
Issue #17
[WBS 2.3] Implementar generador PWM
```

Cada Issue deberá procurar incluir:

- Título claro
- Referencia a la actividad de la WBS
- Descripción
- Responsable
- Criterio para considerarlo terminado
- Sprint correspondiente
- Etiquetas cuando sean útiles
- Evidencia o comentarios relevantes

Un Issue no debe limitarse a decir:

```
Hacer PWM
```

Debe describir suficientemente el trabajo esperado.

7.3. Branches

Cada actividad relevante deberá desarrollarse en una rama independiente.

Una convención recomendada es:

```
feature/numero-issue-descripcion
```

Ejemplo:

```
feature/17-pwm-generator
```

Otros prefijos aceptables pueden ser:

```
feature/
fix/
docs/
test/
hardware/
```

según la naturaleza del cambio.

7.3.1. Ejemplos

```
feature/21-temperature-sensor
hardware/24-buck-pcb
docs/07-update-risk-register
fix/35-pwm-duty-limit
test/41-power-stage-test
```

7.4. Commits

Los commits deberán representar cambios identificables.

Ejemplos adecuados:

```
#17 Add initial PWM implementation
#17 Configure switching frequency
#17 Add duty-cycle limits
```

Ejemplos **NO** adecuados:

```
cambio
otro cambio
ya quedo
final
final2
ahora si final
```

Regla de trabajo

La calidad del trabajo no será evaluada contando commits. Un número elevado de commits no implica necesariamente una contribución mayor.

7.5. Pull Requests

Cuando una actividad esté lista para ser integrada, el responsable deberá crear un **Pull Request** hacia **main**.

Ejemplo:

```
[WBS 2.3] Implement PWM generator
```

El Pull Request deberá indicar:

- Qué se realizó
- Qué Issue resuelve
- Qué archivos fueron modificados
- Qué pruebas o verificaciones se hicieron
- Qué aspectos quedan pendientes, si existen

Siempre que sea posible, otro integrante deberá revisar el cambio antes de integrarlo.

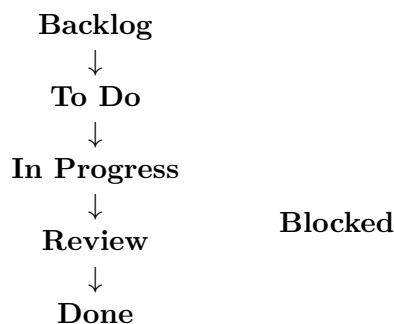
Después de la revisión:

Pull Request → Merge a main → Issue cerrado

8. GitHub Project

Cada equipo deberá mantener un tablero de seguimiento mediante **GitHub Projects** o una herramienta equivalente autorizada.

Como mínimo, deberá manejar los estados:



Blocked se utilizará cuando una actividad no pueda continuar por alguna dependencia, recurso, decisión, falla o restricción.

9. SPRINT I: Planificación de la construcción

Objetivo

Convertir el diseño recibido en un proyecto de construcción ejecutable, estableciendo qué se construirá, qué trabajo será necesario, quién realizará cada actividad, qué riesgos existen y cuándo deberá ejecutarse el trabajo.

El Sprint I no consiste simplemente en llenar formatos.
Los documentos deberán elaborarse a partir del proyecto real.

9.1. Documentos obligatorios del Sprint I

La carpeta:

```
/project-management
```

deberá contener cinco instrumentos obligatorios:

1. Project Charter.
2. Work Breakdown Structure (WBS).
3. RACI Matrix.
4. Risk Register.
5. Project Schedule.

Es decir:

```
/project-management
  /charter
  /wbs
  /raci
  /risks
  /schedule
```

9.2. Documento 1: Project Charter

El **Project Charter** formaliza el proyecto de construcción.

Deberá permitir responder: ¿Qué proyecto tenemos y bajo qué condiciones iniciaremos su construcción?

Importante

El archivo con la plantilla del documento Project Charter lo proporcionará el profesor. El objetivo deberá formularse con la metodología SMART: **S**pecific, **M**easurable, **A**ttainable, **R**elevant, and **T**ime-bound.

9.3. Documento 2: Work Breakdown Structure (WBS)

La **WBS** descompone el proyecto en tareas manejables.

Deberá responder: ¿Qué trabajo debemos realizar para materializar la solución?

Importante

El archivo con la plantilla del documento "WBS" lo proporcionará el profesor.

9.4. Documento 3: RACI Matrix

La matriz **RACI** permite asignar responsabilidades.

Importante

El archivo con la plantilla del documento RACI lo proporcionará el profesor. RACI no significa que cada integrante mantenga exactamente el mismo rol durante todo el proyecto.

9.5. Documento 4: Risk Register

El **Risk Register** deberá responder: ¿Qué acontecimientos o condiciones podrían afectar la ejecución del proyecto y qué haremos si ocurren?

Importante

El archivo con la plantilla del documento Risk Register lo proporcionará el profesor.

9.6. Documento 5: Project Schedule

El **Project Schedule** convierte la WBS en una planeación temporal.

Deberá responder: ¿Cuándo y en qué orden deberá realizarse el trabajo?

Importante

El diagrama de Gantt constituye la planeación formal del proyecto. El seguimiento cotidiano del trabajo se realizará principalmente mediante GitHub Projects. El archivo con la plantilla del diagrama de Gantt lo proporcionará el profesor.

10. Actividades del Sprint I en GitHub

La propia elaboración de la planeación deberá dejar evidencia en GitHub.

Por ejemplo:

Issue #1

Complete Project Charter

Issue #2

Develop Work Breakdown Structure

Issue #3

Create RACI Matrix

Issue #4
Prepare Risk Register

Issue #5
Prepare Project Schedule

Los responsables deberán trabajar mediante ramas y Pull Requests.
Esto permitirá practicar el flujo de trabajo antes de iniciar la construcción técnica.

10.1. Relación entre WBS, RACI y GitHub

La documentación del proyecto no deberá mantenerse separada de la ejecución.
Una actividad de la WBS deberá poder relacionarse con un responsable y con evidencia de ejecución.
Por ejemplo:

WBS 2.3 → RACI → Issue #17 → Branch → Commits → Pull Request

Ejemplo:

WBS	Actividad	Responsable	GitHub
2.3	Implementación del generador PWM	María	Issue #17
2.4	Integración del sensor de temperatura	Pedro	Issue #21
3.1	Diseño de la tarjeta PCB	Ana	Issue #24

De esta forma puede establecerse:

Planeación ↔ Responsabilidad ↔ Evidencia

11. Entrega del Sprint I

El resultado final del Sprint I será:

Project Construction Plan — Baseline v1.0

El equipo deberá crear una etiqueta:

sprint-1-plan-baseline-v1.0

Esta versión representará:

“Éste es el plan con el que inicialmente pretendíamos ejecutar la construcción.”

Entrega

El Sprint I deberá incluir, como mínimo:

1. Repositorio GitHub configurado
2. Integrantes trabajando con cuentas individuales
3. README principal
4. Design Baseline
5. Etiqueta `design-baseline-v1.0`
6. Project Charter
7. WBS

- 8. RACI
- 9. Risk Register
- 10. Project Schedule/Gantt
- 11. GitHub Project
- 12. Issues correspondientes al trabajo
- 13. Historial de commits y Pull Requests
- 14. Etiqueta `sprint-1-plan-baseline-v1.0`

12. SPRINT II: Seguimiento, evaluación y adaptación

Objetivo

Contrastar el plan inicial con lo realmente ejecutado, identificar desviaciones y actualizar la ruta de construcción.

Durante el Sprint II se realizará la mayor parte de la ejecución técnica del proyecto.

El equipo deberá:

- Construir
- Programar
- Fabricar
- Integrar
- Implementar
- Verificar parcialmente
- Registrar incidencias
- Actualizar riesgos
- Documentar decisiones
- Controlar modificaciones

según la naturaleza de cada proyecto.

12.1. Seguimiento continuo

El Project Leader deberá procurar que el estado mostrado en GitHub sea consistente con el estado real del proyecto.

Una actividad no deberá aparecer como *Done* si todavía no está terminada.

De igual forma, una actividad bloqueada deberá registrarse como tal.

12.2. Reuniones del equipo

Durante las reuniones de seguimiento, cada integrante deberá poder responder:

1. ¿Qué terminé?
2. ¿Qué estoy realizando actualmente?
3. ¿Qué actividad sigue?
4. ¿Qué problema o dependencia me está bloqueando?
5. ¿Necesito alguna decisión, recurso o intervención?

Estas preguntas no deberán contestarse únicamente de manera verbal.

Cuando sea posible, deberán estar respaldadas por evidencia en el repositorio.

12.3. Revisión con el mentor

En las reuniones con el mentor, el equipo deberá presentar:

- Avance técnico
- Hitos alcanzados
- Bloqueos
- Riesgos
- Decisiones pendientes
- Necesidades de recursos
- Modificaciones relevantes

El mentor participa como Project Sponsor y Technical Mentor, pero no debe sustituir la responsabilidad de organización del equipo.

12.4. Project Review

Durante una revisión del proyecto, el equipo deberá estar preparado para responder preguntas como:

- ¿Qué estaba planeado?
- ¿Qué se ha realizado?
- ¿Dónde está la evidencia?
- ¿Quién era responsable?
- ¿Qué actividad se retrasó?
- ¿Por qué ocurrió la desviación?
- ¿Qué riesgo se materializó?
- ¿Qué decisión se tomó?
- ¿Qué impacto tuvo?
- ¿Qué debe modificarse en el plan?

12.5. Cambios durante la construcción

Modificar el plan no constituye, por sí mismo, un error.

En proyectos reales, aparecen:

- Problemas técnicos
- Retrasos
- Nuevas necesidades
- Restricciones
- Resultados inesperados
- Cambios de componentes
- Modificaciones de alcance

Lo importante es que los cambios sean:

1. Identificados
2. Justificados
3. Documentados
4. Incorporados al estado actual del proyecto

Importante

La finalidad de establecer una baseline no es impedir modificaciones, sino permitir saber posteriormente **qué cambió respecto de lo planeado**.

12.6. Documentos vivos durante el Sprint II

Los documentos del Sprint I no se abandonan.

Durante el Sprint II deberán actualizarse cuando sea necesario:

- WBS
- RACI
- Risk Register
- Project Schedule

El Project Charter solamente deberá modificarse cuando exista un cambio que realmente afecte aspectos fundamentales del proyecto.

13. Entrega del Sprint II

La entrega principal será:

Sprint II Progress Review

Deberá mostrar:

- | | |
|------------------------------------|---|
| 1. Actividades planeadas | 10. Decisiones |
| 2. Actividades realizadas | 11. Cambios |
| 3. Hitos alcanzados | 12. Justificación de los cambios |
| 4. Actividades pendientes | 13. Impacto sobre el proyecto |
| 5. Desviaciones | 14. WBS actualizada |
| 6. Recursos planeados y utilizados | 15. RACI actualizada cuando corresponda |
| 7. Riesgos materializados | 16. Risk Register actualizado |
| 8. Nuevos riesgos | 17. Schedule actualizado |
| 9. Incidencias | 18. Ruta restante hacia el Sprint III |

Entrega

La evidencia del Sprint II deberá encontrarse tanto en la documentación formal como en el historial real del repositorio: Issues, ramas, commits, Pull Requests, archivos, versiones y tablero de seguimiento.

Al finalizar podrá establecerse una versión:

```
sprint-2-review-v1.0
```

14. SPRINT III: Consolidación y transición hacia Pruebas

Objetivo

Establecer objetivamente el estado final alcanzado durante Construcción, consolidar la documentación y preparar la transferencia de la solución al Laboratorio Abierto: Pruebas.

El Sprint III no exige necesariamente que el proyecto esté construido al 100 %. Sin embargo, sí exige que pueda determinarse con claridad:

- | | | |
|-----------------------|-----------------------------------|---------------------------------|
| ▪ Qué está construido | ▪ Qué está parcialmente terminado | ▪ Qué limitaciones existen |
| ▪ Qué funciona | ▪ Qué permanece pendiente | ▪ Qué deberá realizarse después |
| ▪ Qué está integrado | | |

14.1. Consolidación

Durante este Sprint deberá prestarse especial atención a:

- | | |
|---------------------------------------|-----------------------------|
| ▪ Integración | ▪ Actualización de la BOM |
| ▪ Documentación | ▪ Registro de pendientes |
| ▪ Consistencia del repositorio | ▪ Defectos conocidos |
| ▪ Identificación de versiones | ▪ Estado de cada subsistema |
| ▪ Actualización de planos y diagramas | ▪ Preparación para pruebas |
| ▪ Actualización de software | |

14.2. Handover

El concepto principal del cierre será:

Construction Handover

El objetivo es dejar el proyecto en condiciones tales que otra persona, equipo o laboratorio pueda comprender:

- Qué fue construido
- Cómo está configurado
- Qué versión corresponde al cierre
- Qué funciona
- Qué falta
- Qué debe probarse
- Qué riesgos o restricciones continúan abiertos

14.3. Construction Handover Package

La entrega final deberá contener un:

Construction Handover Package

que incluya, según corresponda:

1. Estado de la solución

Identificación objetiva de:

- Componentes terminados
- Componentes parcialmente terminados
- Componentes no iniciados
- Integración alcanzada

2. Evidencia técnica

Puede incluir:

- Prototipo físico
- Planos
- Modelos
- Resultados parciales
- Software
- PCB
- Fotografías
- Demostración funcional
- Firmware
- Modelos CAD
- Video

3. Historial del proyecto

Deberá poder identificarse:

- Plan inicial
- Decisiones
- Incidencias
- Evolución de la solución
- Modificaciones
- Riesgos
- Cambios de responsabilidad

4. Trabajo pendiente

Toda actividad pendiente deberá quedar identificada.
No se deberá ocultar el trabajo incompleto.

5. Preparación para pruebas

Se recomienda presentar una tabla como:

Elemento	Estado	Evidencia	Listo para Pruebas
PCB de potencia	Completo	Revisión 3	Sí
Sensor	Completo	Revisión 2	Sí
Firmware	Parcial	Versión 0.8	Parcialmente
Carcasa	Pendiente	—	No

15. Versión final del Sprint III

Al concluir la materia deberá establecerse una etiqueta, por ejemplo:

```
construction-handover-v1.0
```

Esta versión deberá representar el estado exacto de la solución que se transfiere a Pruebas.

16. Las tres versiones principales del proyecto

Al finalizar la materia deberán poder identificarse, como mínimo, tres momentos:

Versión	Significado
<code>design-baseline-v1.0</code>	Diseño recibido antes de iniciar la construcción.
<code>sprint-1-plan-baseline-v1.0</code>	Plan inicial mediante el cual el equipo pretendía realizar la construcción.
<code>construction-handover-v1.0</code>	Estado real alcanzado al finalizar la Construcción y transferido a Pruebas.

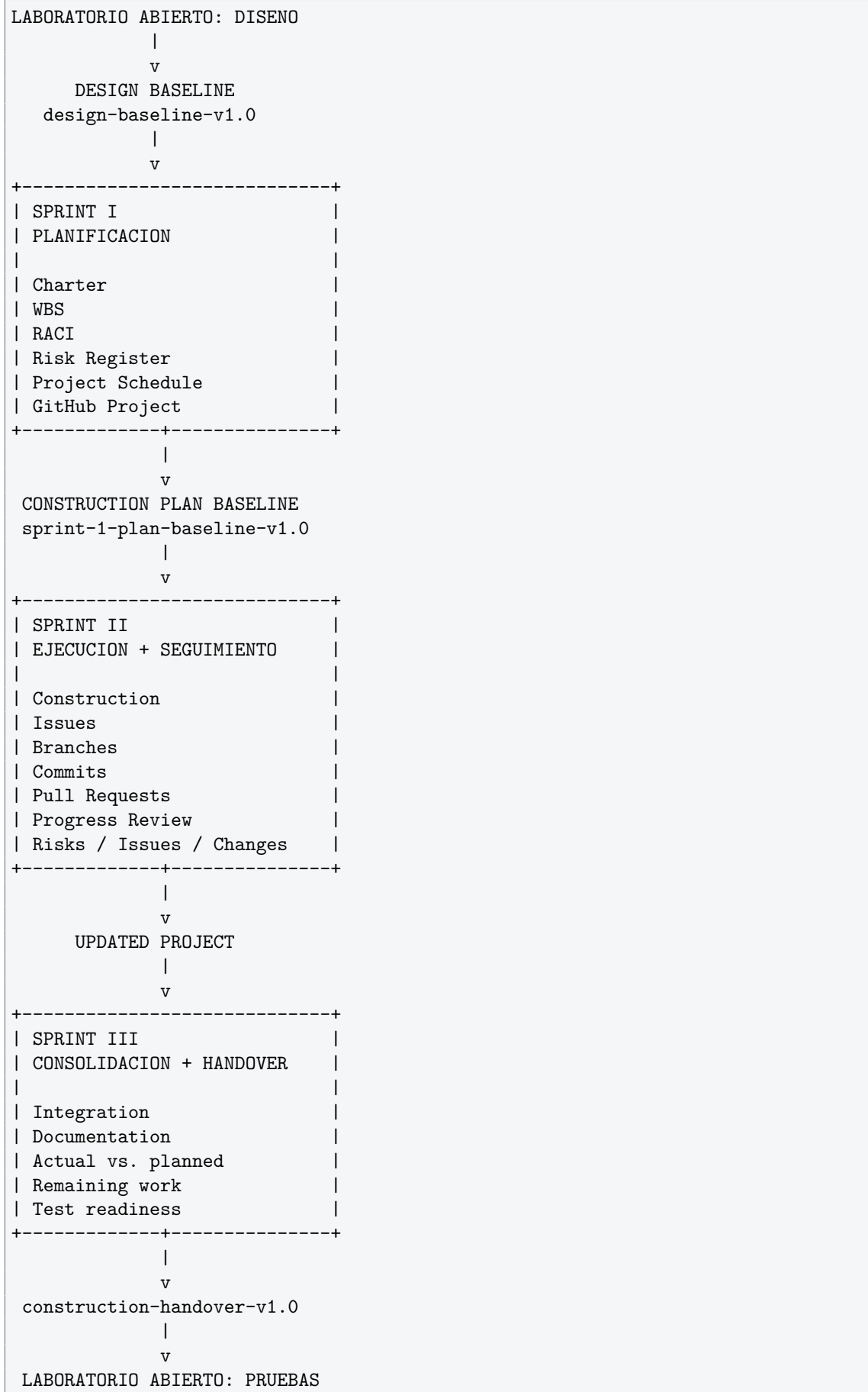
Esto permitirá estudiar la evolución:

```
Design Baseline → Construction Plan Baseline → Actual Construction
```

16.1. Resumen operativo de los tres Sprints

Etapas	Pregunta central	Trabajo principal	Resultado
Design Baseline	¿Qué diseño recibimos?	Organizar y congelar el estado inicial.	<code>design-baseline-v1.0</code>
Sprint I	¿Cómo pensamos construirlo?	Charter, WBS, RACI, riesgos, Schedule y configuración de GitHub.	<code>sprint-1-plan-baseline-v1.0</code>
Sprint II	¿Qué está ocurriendo realmente?	Construcción, seguimiento, actualización, control de riesgos, incidencias y cambios.	Sprint II Progress Review
Sprint III	¿Qué logramos construir y qué transferimos?	Consolidación, documentación, estado real y preparación para Pruebas.	<code>construction-handover-v1.0</code>

16.2. Mapa general del proceso



17. Reglas generales del repositorio

Todos los proyectos deberán respetar las siguientes reglas:

1. Existirá un repositorio principal por proyecto.
2. Cada estudiante utilizará su propia cuenta.
3. La rama **main** representará el estado integrado del proyecto.
4. Después de establecer la Design Baseline, no se desarrollará directamente sobre **main**.
5. Las actividades relevantes deberán estar asociadas a Issues.
6. Los Issues deberán relacionarse, cuando sea posible, con la WBS.
7. El trabajo se desarrollará mediante branches.
8. Los cambios relevantes deberán integrarse mediante Pull Requests.
9. Los commits deberán describir cambios concretos.
10. El GitHub Project deberá reflejar el estado real del trabajo.
11. Los documentos de gestión deberán mantenerse actualizados.
12. Cada Sprint deberá dejar una versión identificable.
13. El repositorio deberá contener evidencia suficiente para reconstruir la evolución del proyecto.

17.1. Convención recomendada de nombres

Se recomienda utilizar nombres consistentes y evitar:

```
final.docx
final_final.docx
ahora_si_final.xlsx
ultima_version2.pdf
```

Ejemplos adecuados:

```
Project_Charter_v1.0.pdf
WBS_v1.0.xlsx
RACI_v1.0.xlsx
Risk_Register_v1.0.xlsx
Project_Schedule_v1.0.xlsx
```

Cuando un documento sea actualizado:

```
Risk_Register_v1.1.xlsx
Project_Schedule_v1.2.xlsx
```

El historial de Git complementará este esquema de versiones.

17.2. Qué significa “terminado”

Una actividad no se considera terminada únicamente porque alguien diga que la terminó.

Dependiendo de la tarea, “Done” puede requerir:

- | | | |
|--------------------------|-----------------------------|-------------------------|
| ▪ Archivo generado | ▪ Revisión completada | ▪ Pull Request aprobado |
| ▪ Código integrado | ▪ Documentación actualizada | ▪ Evidencia almacenada |
| ▪ Dispositivo ensamblado | ▪ Verificación realizada | |

Por ello, cada Issue deberá tener un criterio razonable de finalización.

17.3. Uso de evidencia

La evidencia puede incluir:

- Archivos
- Fotografías
- Capturas
- Videos
- Resultados
- Diagramas
- Mediciones
- Registros
- Versiones
- Comentarios técnicos
- Pruebas preliminares
- Enlaces a Issues o Pull Requests

La evidencia deberá permitir relacionar una afirmación con un resultado observable.

17.4. Bitácora de decisiones

Las decisiones importantes deberán documentarse.

Cuando una decisión afecte significativamente:

- Diseño
- Materiales
- Tecnología
- Alcance
- Tiempo
- Presupuesto
- Integración
- Responsabilidades

deberá existir una breve justificación.

Una estructura sencilla puede ser:

ID	Fecha	Decisión	Justificación	Responsable
D-01	12/09	Cambiar controlador	El dispositivo originalmente seleccionado no está disponible	Project Leader / Mentor

Esta bitácora puede mantenerse dentro de:

/project-management/decisions

si el proyecto lo requiere.

18. Actores y Roles

18.1. Qué se espera del Project Leader

El Project Leader deberá procurar que:

- Exista una planeación
- Las actividades tengan responsables
- El tablero refleje el estado real
- Los bloqueos sean visibles
- Las decisiones relevantes se documenten
- Los cambios se incorporen a los documentos
- Se realicen las revisiones
- El equipo pueda presentar evidencia
- Se mantenga coordinación con el mentor

No es responsabilidad del Project Leader realizar personalmente las actividades que otro integrante dejó pendientes.

18.2. Qué se espera del Project Team

Cada integrante deberá:

- Conocer sus responsabilidades
- Revisar la WBS y la RACI
- Mantener actualizadas sus actividades
- Utilizar su propia cuenta de GitHub
- Trabajar mediante las ramas correspondientes
- Realizar commits identificables
- Crear o participar en Pull Requests
- Informar bloqueos
- Documentar resultados
- Participar en las revisiones
- Entregar evidencia de su trabajo

18.3. Qué se espera del mentor

El mentor participa principalmente en:

- Orientación técnica
- Revisión de decisiones relevantes
- Discusión de riesgos técnicos
- Validación de cambios significativos
- Apoyo ante bloqueos
- Seguimiento del estado general

El mentor no sustituye al Project Leader ni al Project Team.

19. Qué se espera durante una revisión

En cualquier revisión, el equipo deberá poder mostrar el proyecto directamente desde sus herramientas de trabajo.

Se podrá solicitar, por ejemplo:

- Abrir el GitHub Project
- Mostrar una actividad
- Identificar su responsable
- Abrir el Issue
- Mostrar la branch
- Revisar commits
- Abrir un Pull Request
- Localizar el elemento correspondiente en la WBS
- Mostrar el riesgo relacionado
- Comparar Schedule planeado y actualizado
- Mostrar evidencia técnica

Regla de trabajo

La documentación deberá reflejar el proyecto real. No deberá actualizarse únicamente minutos antes de una revisión para aparentar un estado que no corresponde al trabajo realizado.

19.1. Trazabilidad individual

Una finalidad central del esquema de trabajo es identificar la aportación de cada integrante. La contribución individual se analizará mediante diversas fuentes de evidencia.

RACI + Issues + Branches + Commits + Pull Requests + Artefactos técnicos

Por ejemplo, si un estudiante aparece como Responsable de:

WBS 2.3 — Implementar generador PWM

deberá existir una relación razonable con:

- Issue asignado
- commits
- Evidencia técnica
- o modificados
- Rama de trabajo
- Pull Request
- Archivos producidos

20. Listados de verificación

20.1. Sprint I

Antes de entregar el Sprint I, el equipo deberá verificar:

- ☐ Todos los integrantes tienen una cuenta individual de GitHub.
- ☐ Todos los integrantes tienen acceso al repositorio.
- ☐ El repositorio tiene README.
- ☐ Existe la carpeta `/design-baseline`.
- ☐ Se identificó el estado recibido desde Diseño.

- ☐ Existe la etiqueta **design-baseline-v1.0**.
- ☐ Se seleccionó Project Leader.
- ☐ Se identificó Project Sponsor.
- ☐ Se completó el Project Charter.
- ☐ Se completó la WBS.
- ☐ Se completó la matriz RACI.
- ☐ Se completó el Risk Register.
- ☐ Se completó el Project Schedule.
- ☐ Existe un Gantt o representación equivalente.
- ☐ Las actividades están registradas en GitHub.
- ☐ Existe un GitHub Project.
- ☐ Se utilizaron Issues.
- ☐ Se utilizaron branches.
- ☐ Se realizaron commits identificables.
- ☐ Se utilizaron Pull Requests.
- ☐ La información de GitHub corresponde con los documentos de planeación.
- ☐ Existe la etiqueta **sprint-1-plan-baseline-v1.0**.

20.2. Sprint II

- ☐ El tablero muestra el estado real del proyecto.
- ☐ Las actividades realizadas tienen evidencia.
- ☐ Los integrantes tienen contribuciones identificables.
- ☐ Los bloqueos están documentados.
- ☐ Los riesgos fueron revisados.
- ☐ Los riesgos ocurridos fueron registrados como incidencias.
- ☐ Los nuevos riesgos fueron incorporados.
- ☐ Las decisiones importantes están documentadas.
- ☐ La WBS fue actualizada cuando fue necesario.
- ☐ La RACI fue actualizada cuando fue necesario.
- ☐ El Schedule fue actualizado.
- ☐ Se identificaron desviaciones respecto de la baseline.
- ☐ Se estableció la ruta restante al Sprint III.

20.3. Sprint III

- ☐ Se determinó el estado real de cada componente o subsistema.
- ☐ Se documentaron los principales cambios.
- ☐ Se consolidó la documentación técnica.
- ☐ Se identificaron actividades pendientes.
- ☐ Se identificaron defectos o limitaciones conocidas.
- ☐ La BOM está actualizada cuando aplica.
- ☐ Los planos y diagramas corresponden con la versión construida.
- ☐ El software o firmware corresponde con la versión entregada.
- ☐ Se identificó qué está listo para Pruebas.
- ☐ Se identificó qué no está listo para Pruebas.
- ☐ Se establecieron las acciones siguientes.

- ☐ Existe suficiente evidencia de la contribución de cada integrante.
- ☐ Se creó la versión `construction-handover-v1.0`.

21. Principios del laboratorio

La forma de trabajo de la materia se resume en los siguientes principios:

1. **Planear antes de ejecutar.**
2. **Asignar responsabilidades antes de evaluar resultados.**
3. **Mantener trazabilidad entre planeación y ejecución.**
4. **Utilizar control de versiones desde el inicio.**
5. **Documentar el trabajo conforme ocurre.**
6. **No ocultar desviaciones, problemas o trabajo pendiente.**
7. **Distinguir entre el plan y el estado real.**
8. **Justificar los cambios importantes.**
9. **Mantener evidencia individual y colectiva.**
10. **Entregar una solución que pueda ser comprendida y continuada por otros.**

22. Mensaje final al equipo

En esta materia no se espera únicamente un resultado técnico.

Se espera que cada equipo pueda responder de manera documentada:

- ¿Qué recibimos?
- ¿Qué decidimos construir?
- ¿Cómo planeamos hacerlo?
- ¿Quién era responsable de cada actividad?
- ¿Qué recursos necesitábamos?
- ¿Qué riesgos identificamos?
- ¿Qué hicimos realmente?
- ¿Quién realizó cada aportación?
- ¿Qué problemas aparecieron?
- ¿Qué decisiones tomamos?
- ¿Qué cambió respecto del plan?
- ¿Qué logramos construir?
- ¿Qué quedó pendiente?
- ¿Qué estamos entregando al siguiente laboratorio?

Objetivo

Un proyecto bien gestionado no es aquel en el que nunca aparecen problemas o cambios. Es aquel en el que el equipo puede identificar su estado real, responder ante las condiciones que aparecen y mantener trazabilidad suficiente para explicar cómo evolucionó el trabajo desde el diseño inicial hasta el resultado construido.